

EX.NO: 6	ANGULAR: CREATE USER REGISTRATION FORM USING REACTIVE OR TEMPLATE FORM WITH VALIDATION
DATE:	

QUESTION:

Create user registration form using reactive or template form with validation using angular.

AIM:

To create user registration form using reactive or template form with validation using angular.

ALGORITHM:

- 1) Import ReactiveFormsModule in app.module.ts.
- 2) Create a FormGroup in the component with controls for name, email, password.
- 3) Add validators (required, email, minLength).
- 4) Bind the form to the template using [formGroup].
- 5) Use FormControlName for each input field.
- 6) Display error messages when validation fails.
- 7) Add a submit button with (ngSubmit) event.
- 8) Check form.valid before processing data.
- 9) Print submitted values in console or UI.
- 10) Reset form after successful submission.

PROGRAM:**app.module.ts**

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { ReactiveFormsModule } from '@angular/forms';
import { AppComponent } from './app.component';
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, ReactiveFormsModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

app.component.ts

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  registrationForm = new FormGroup({
    name: new FormControl("", Validators.required),
    email: new FormControl("", [Validators.required, Validators.email]),
    password: new FormControl("", [Validators.required, Validators.minLength(6)])
  });

  onSubmit() {
    if (this.registrationForm.valid) {
      console.log(this.registrationForm.value);
      alert('Registration Successful!');
      this.registrationForm.reset();
    }
  }
}

```

app.component.html

```

<form [formGroup]="registrationForm" (ngSubmit)="onSubmit()">
  <label>Name:</label>
  <input type="text" formControlName="name" />
  <div *ngIf="registrationForm.get('name')?.invalid && registrationForm.get('name')?.touched">
    Name is required
  </div>

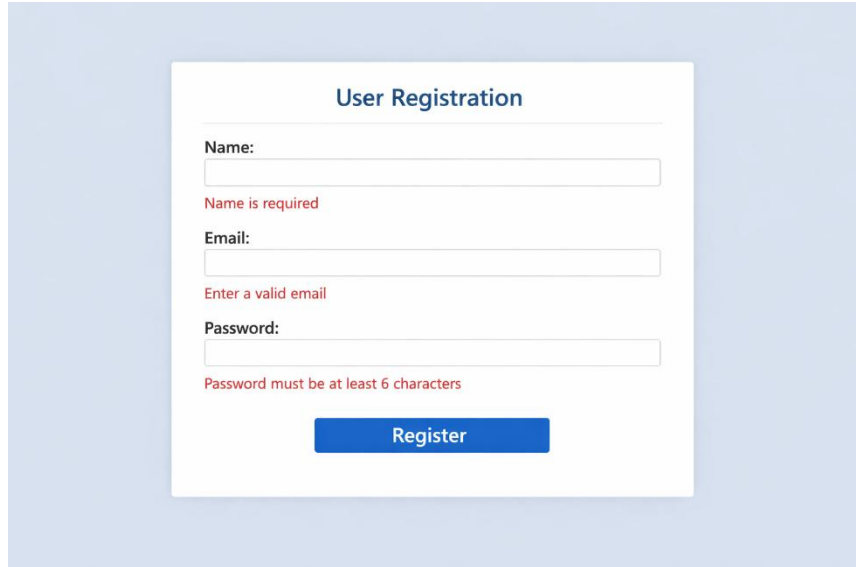
  <label>Email:</label>
  <input type="email" formControlName="email" />
  <div *ngIf="registrationForm.get('email')?.invalid && registrationForm.get('email')?.touched">
    Enter a valid email
  </div>

  <label>Password:</label>
  <input type="password" formControlName="password" />
  <div *ngIf="registrationForm.get('password')?.invalid &&
  registrationForm.get('password')?.touched">
    Password must be at least 6 characters
  </div>

  <button type="submit" [disabled]="registrationForm.invalid">Register</button>
</form>

```

OUTPUT:

A screenshot of a web form titled "User Registration". The form is white with a blue border and is set against a light blue background. It contains three input fields: "Name:", "Email:", and "Password:". Below the "Name:" field, there is a red error message "Name is required". Below the "Email:" field, there is a red error message "Enter a valid email". Below the "Password:" field, there is a red error message "Password must be at least 6 characters". At the bottom of the form, there is a blue button labeled "Register".

User Registration

Name:

Name is required

Email:

Enter a valid email

Password:

Password must be at least 6 characters

Register

Mark Allocation		
Particulars	Marks Allotted	Marks Awarded
Program/Simulation	40	
Program Execution	30	
Result	20	
Viva Voce	10	
Total	100	

Faculty Signature

RESULT:

Thus, the angular dynamic reactive form works as intended, allowing users to add/remove fields and submit data has been executed successfully